

DOSEdit — A Visual Basic for DOS / QuickBASIC-Style Multi-Window Text Editor

Build specification & implementation hand-off Target implementer: Claude Code (or any Go developer) Language / stack: **Go + tcell + tview + winman** Scope: **Multi-window console text editor only** — authentic VBDOS / QuickBASIC look & feel, no compile/run. Version of this spec: 1.1 — palette corrected against real VBDOS screenshots (grey desktop, magenta title bars, cyan status bar, vertical dialog buttons).

0. How to use this document

This is a complete, self-contained build brief. Everything needed to implement the app is here: the visual design system, component-by-component behaviour, the full keyboard map, the file/document model, the Go project layout, dependency list, build commands, a phased milestone plan, acceptance criteria, and starter code skeletons in the appendix.

Two kinds of "screenshots" are included:

- **Rendered PNG mockups** (`images/*.png`) — for the human reviewer, showing the exact target colours and look.
- **ASCII layout mockups** (inline code blocks) — these are the *authoritative* layout reference. They specify character positions, box-drawing glyphs, and alignment. When in doubt, the ASCII mockups win, because they are unambiguous about cells.

Build order: follow **§11 Milestones**. Do not attempt everything at once. Each milestone ends in a runnable binary.

1. Summary

DOSEdit is a cross-platform terminal application that recreates the early-1990s **Visual Basic for DOS 1.0 / Microsoft QuickBASIC 4.5** integrated editor experience: a textured **grey** desktop, a light-grey CUA menu bar with white mnemonic letters, multiple movable / resizable / overlapping MDI child windows with **magenta active title bars**, modal dialog boxes with drop shadows, and a **cyan** function-key status bar.

It is a **text editor**, not an IDE. There is no interpreter, compiler, debugger, or "Run". Files are plain text (any extension; `.bas` is the default filter only because it fits the theme).

2. Goals & non-goals

Goals

1. Pixel-faithful recreation of the VBDOS / QuickBASIC visual style (see §4).
2. True **MDI**: many child windows at once; each is independently movable, resizable, maximizable, and closable, and they can overlap with a correct Z-order.
3. A genuinely usable plain-text editor inside each window: cursor navigation, selection, clipboard, find/replace, undo/redo, open/save.
4. Full keyboard-first operation (CUA shortcuts + classic function keys). Mouse is supported but never required.
5. One static binary per OS. Runs on Linux, macOS, and Windows terminals.

Non-goals (explicitly out of scope for v1)

- No code execution / build / run / debug.
- No language services (autocomplete, error checking). Syntax highlighting is an **optional stretch** (§7.4), default **off**.
- No project/workspace concept beyond "a set of open files".
- No split views inside a single window. One buffer = one window.
- No mouse-driven menu *required*; mouse menu support is a stretch goal.

3. Target platform & technology stack

Concern	Choice	Notes
---------	--------	-------

Language	Go 1.22+	Use modules.
Terminal cell engine	github.com/gdamore/tcell/v2	Colours, key events, mouse, screen.
Widget toolkit	github.com/rivo/tview	App loop, Box, TextArea/TextView, Form, InputField, Modal, List, Flex, Pages.
MDI window manager	github.com/epiclabs-io/winman	Draggable / resizable / overlapping windows with Z-index + title-bar buttons. This is the key dependency — base tview has <i>no</i> window manager.

Important framework facts (verified — do not assume otherwise)

- **tview has no built-in menu bar and no built-in MDI window manager.** The menu bar, the dropdown menus, and the function-key status bar must be implemented as **custom primitives** (embed `tview.Box`, implement `Draw` and `InputHandler`). See appendix B.
- MDI windows come from **winman**, used on top of tview. The window manager primitive becomes (or is wrapped by) the application's root layout. Each `winman` window's content (`SetRoot`) is one editor primitive.
- `winman.NewWindow()` supports `.SetDraggable(true)`, `.SetResizable(true)`, `.SetModal(...)`, `.AddButton(...)` (title-bar buttons), Z-index control, and `.Show().Hide()`.
- If `epiclabs-io/winman` proves stale at build time, the API-compatible fork github.com/r3ap3r2004/winman is an acceptable substitute. Verify `go get` succeeds before committing to one.

4. Visual design system

4.1 Screen geometry

- Logical screen is the full terminal. Reference design is **80 × 25** but the layout must reflow to any size $\geq 60 \times 15$.
- Row 0 = menu bar (always). Row N-1 = status bar (always). Rows 1..N-2 = the desktop (the MDI area).

4.2 The DOS 16-colour palette (authoritative RGB)

Define these once as `tcell.Color` values via `tcell.NewRGBColor(r,g,b)`. **Do not** rely on the terminal's named ANSI colours — set RGB explicitly so the look is identical everywhere.

#	Name	Hex	RGB
0	Black	#000000	0,0,0
1	Blue	#0000AA	0,0,170
2	Green	#00AA00	0,170,0
3	Cyan	#00AAAA	0,170,170
4	Red	#AA0000	170,0,0
5	Magenta	#AA00AA	170,0,170
6	Brown	#AA5500	170,85,0
7	Light Gray	#AAAAAA	170,170,170
8	Dark Gray	#555555	85,85,85
9	Light Blue	#5555FF	85,85,255
10	Light Green	#55FF55	85,255,85
11	Light Cyan	#55FFFF	85,255,255
12	Light Red	#FF5555	255,85,85
13	Light Magenta	#FF55FF	255,85,255
14	Yellow	#FFFF55	255,255,85
15	White	#FFFFFF	255,255,255

4.3 Colour assignments per component

These values are taken from real VBDOS screenshots — they are authoritative. The early Turbo-Vision-style guess (blue desktop / grey status bar) was wrong; VBDOS uses a **grey hatched desktop**, **magenta active title bars**, and a **cyan status bar**. The blue only appears *inside* editor windows.

Element	Foreground	Background
Desktop fill (textured )	Black (0)	Light Gray (7) — grey hatch, <i>not</i> blue

Menu bar (normal item)	Black (0)	Light Gray (7)
Menu bar / menu item mnemonic letter	White (15)	(same as item bg)
Menu bar item, open/selected	White (15)	Black (0) (reverse video)
Dropdown menu body	Black (0)	Light Gray (7)
Dropdown highlighted item	White (15)	Black (0) ; mnemonic in Light Cyan (11)
Dropdown disabled item	Dark Gray (8)	Light Gray (7)
Menu / dialog drop shadow	—	solid Black (0) (see §4.5)
Active window frame	White (15)	Blue (1)
Active window title bar	White (15)	Magenta (5, #AA00AA)
Inactive window frame + title bar	Light Gray (7) / Black on grey	Blue (1) frame, Light Gray (7) title
Editor text	White (15)	Blue (1)
Editor selection (reverse)	Blue (1)	Cyan (3)
Text cursor cell	Black (0)	White (15)
Window title-bar buttons   	Black/White	Magenta (5) when active, Light Gray (7) when not
Scrollbar track	Light Blue (9)	Blue (1) (drawn as )
Scrollbar thumb / arrows	Black (0)	Blue (1) (thumb  , arrows )
Dialog body	Black (0)	Light Gray (7)
Dialog title bar	White (15)	Magenta (5)
Dialog input field	White (15)	Black (0); cursor char in Yellow (14)
Dialog list box	Black (0)	White (15)
Dialog list item, selected	White (15)	Black (0) (reverse)
Button face (normal)	Black (0)	Light Gray (7)
Button drop shadow	—	solid Black (0), on right + bottom (3-D raised look)
Button (default / focused)	Black (0)	Light Gray (7) + a black border drawn around it
Status bar	Black (0)	Cyan (3, #00AAAA)
Status bar text / labels	Black (0)	Cyan (3) — uniform, no red labels

Title-bar shorthand: active = magenta bar + white text; inactive = light-grey bar + black text. The window *number* is part of the title:  HELLO.BAS,  GREET.BAS, ... A modified buffer prefixes *:  *HELLO.BAS.

4.4 Box-drawing glyph reference

- **Active window frame:** double line —   (and  for the title separators if used).
- **Inactive window frame & dialogs-inner controls:** single line —  .
- **Dialog outer frame:** double line  .
- **Desktop texture:**  (medium shade) drawn Black-on-Light-Gray, giving the characteristic grey hatch.
- **Scrollbars:** track , thumb , arrows .
- **Title-bar buttons:** close , maximize/restore  / .
- **Menu separators:** a full-width  row, joined with  at the menu edges.

4.5 Shadows

Modal dialogs, open dropdown menus, and dialog buttons cast a one-cell **solid black** shadow on their right and bottom edges (this is what gives buttons their 3-D "raised" look). Implement by forcing those cells' background to Black (0), keeping or blanking the underlying character. Windows (MDI children) do **not** cast shadows; only menus, dialogs, and buttons do.

5. Master layout

Full-screen reference (80×25).  = grey hatched desktop (light-gray bg, black texture). The window title bar ( HELLO.BAS) is **magenta with white text** when active; the editor interior is **blue**. This is the primary screen the user sees.

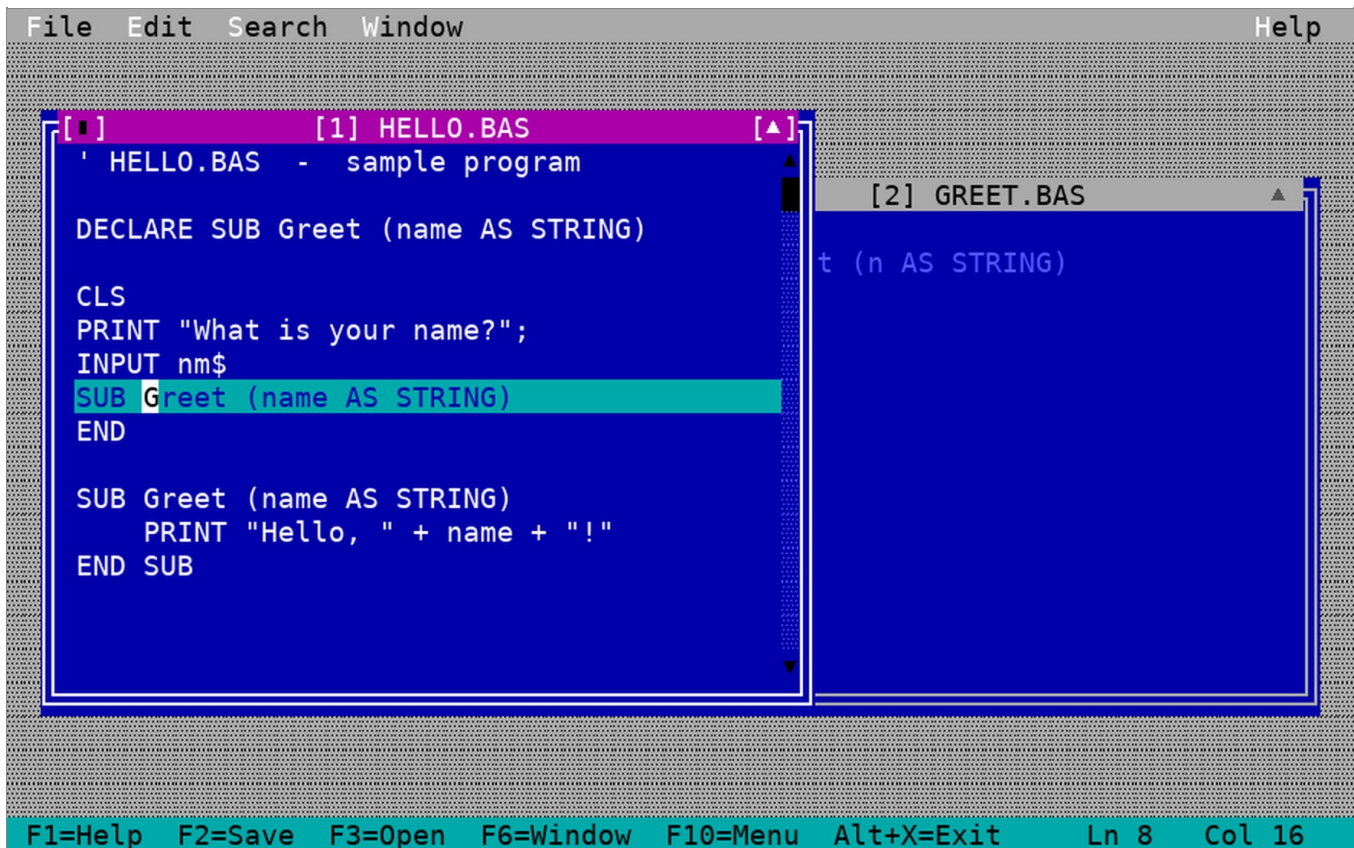


```

[ ] HELLO.BAS [▲]
' HELLO.BAS - sample program
CLS
PRINT "Hello, World!"
END
row 24: status bar (light-gray)
F1=Help F2=Save F3=Open F6=Window F10=Menu Alt+X=Exit Ln 5 Col 1

```

Rendered target:



6. Components

Each component below lists: **purpose** → **layout** → **behaviour** → **keys** → **framework mapping** → **acceptance**.

6.1 Application shell & desktop

Purpose. Owns the tccl screen, the main event loop, global key routing, and the three persistent regions (menu bar, desktop, status bar).

Layout. A `tview.Pages` (or `tview.Flex` vertical) as root: menu bar (height 1, top) · desktop (flex, middle) · status bar (height 1, bottom). The desktop region hosts the **winman window manager** primitive plus a background that paints the `texture`.

Behaviour. * On launch: paint desktop, show one empty `Untitled1` editor window centered, focus it. * Global keys are intercepted at the application level via `tview.Application.SetInputCapture` (or `Box.SetInputHandler` on a wrapper) and dispatched (see §8). Window-local keys fall through to the focused editor. * Resize: reflow menu bar / status bar to full width; clamp any window that now lies outside the desktop back inside it.

Framework mapping. `tview.NewApplication()`, root = vertical `Flex`. Desktop = a custom `Box` that draws the texture in its `Draw` before delegating to the embedded `winman.Manager`. Easiest path: subclass winman's manager draw, or layer a texture `Box` behind the manager using `Pages`.

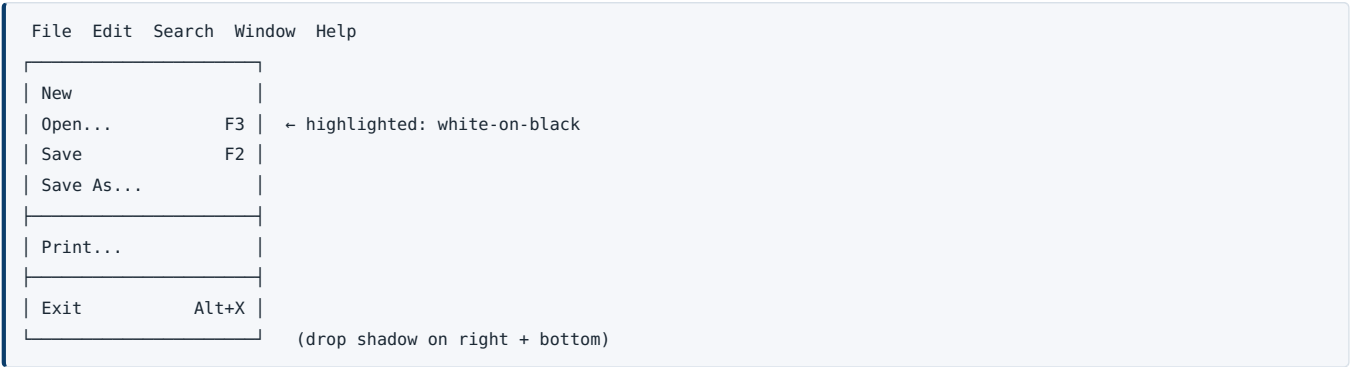
Acceptance. App launches to the master layout, one empty window focused, Alt+X quits cleanly restoring the terminal.

6.2 Menu bar + dropdown menus (custom primitive)

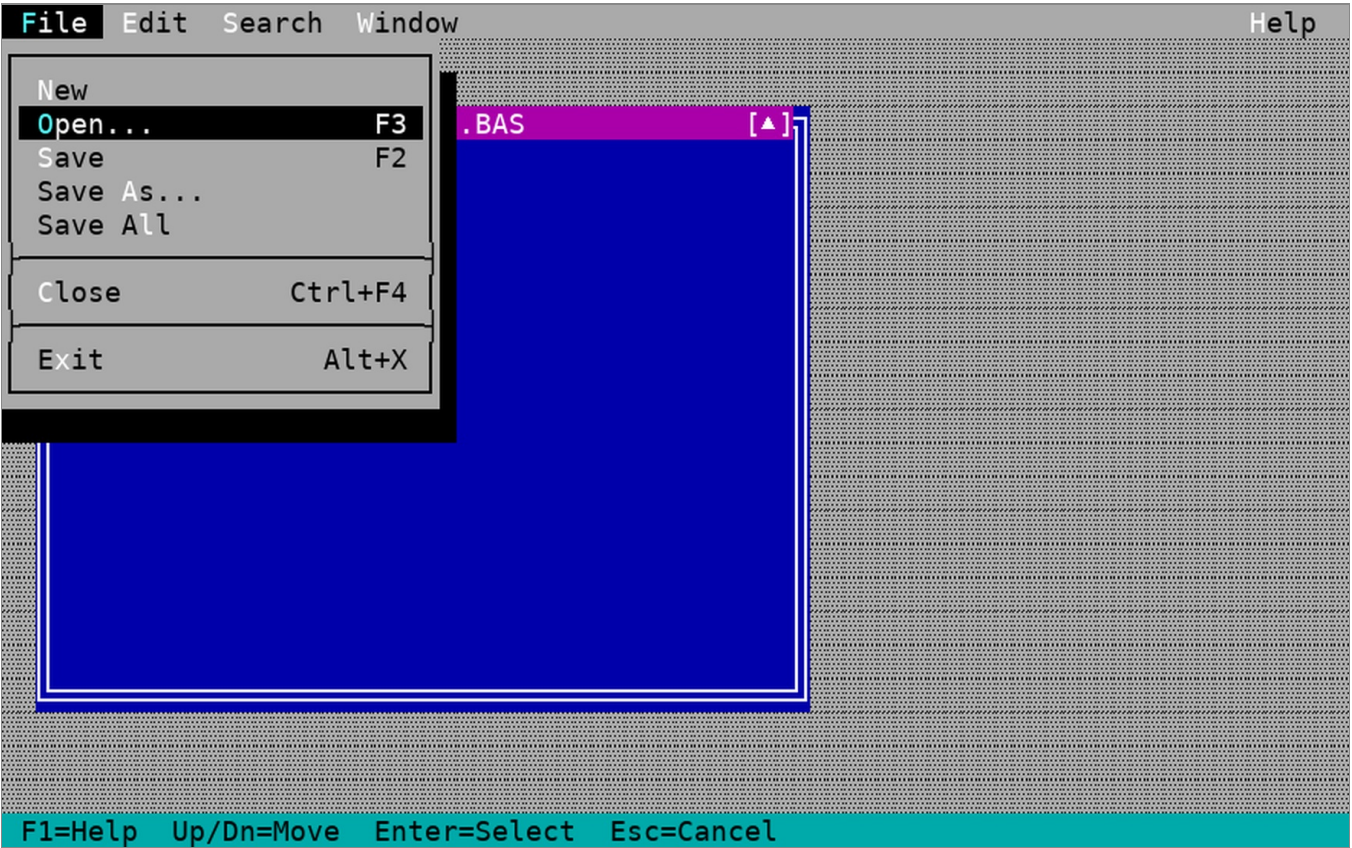
Purpose. Top-row CUA menu; the only always-visible command surface.

Layout. Single row, light-gray. Top-level items left-aligned with 2 spaces of lead and a space between: `File Edit Search Window ...` with `Help` pushed to the right edge. The mnemonic (first letter) is drawn **White** (bright) on the grey bar. When a top-level item is open/active, its cell range is painted **White-on-Black** (reverse video). The dropdown's highlighted row is likewise **White-on-Black**, with its mnemonic in Light Cyan.

Dropdown: a bordered (single-line) light-gray box anchored under its top-level item, wide enough for the longest `label ... shortcut` pair, with a solid-black drop shadow. Item rows: 2-space left pad, label, right-aligned accelerator text (e.g. `F3`, `Alt+X`), 2-space right pad. Separators are a full-width `-` rule joined with `| |`. The highlighted item is **White-on-Black**; disabled items are Dark-Gray.



Rendered target:



Behaviour. * `F10` or `Alt` activates the menu bar (highlights first item). `Alt+<letter>` opens that menu directly (e.g. `Alt+F`). * In the bar: `←/→` move between top-level menus (open menu follows if one is already open); `Enter` / `↓` opens the highlighted menu. * In a dropdown: `↑/↓` move (skipping separators/disabled), `Enter` activates, `Esc` closes the dropdown (back to bar), `Esc` again closes the menu entirely and returns focus to the previous window. `←/→` switch to the adjacent menu. * Typing a menu item's mnemonic letter activates it. * Accelerators shown on the right also work globally without opening the menu (e.g. `F3` opens a file from anywhere).

Framework mapping. Custom primitive `MenuBar` embedding `tview.Box`. The open dropdown is rendered as a separate overlay primitive pushed onto a `tview.Pages` layer above the desktop so it draws atop windows and casts its shadow. Menu model is data-driven (see §7 menu tree); actions are `func()` callbacks.

Acceptance. Every menu in §7 opens, navigates by keyboard and mnemonic, dismisses with Esc, and invokes the correct action; accelerators fire globally.

6.3 MDI window manager & child windows (winman)

Purpose. Hold and manage all open editor windows.

Layout. Each child window has a **double-line blue frame**. The **title bar** (top border row) is filled **magenta with white centered text** when the window is active, and **light-grey with black text** when inactive. The title shows the window number + document name (`[1] HELLO.BAS`, `[2] Untitled`); a `*` precedes the name when the buffer is modified (`[1] *HELLO.BAS`). Title-bar buttons: close `[■]` at the top-left, maximize/restore `[▲]` / `[▼]` at the top-right. A vertical scrollbar on the right inner edge; a horizontal scrollbar on the bottom inner edge when the longest line exceeds the view width.

Behaviour. * **New window** appears slightly offset (cascade) from the last one, sized $\sim\frac{2}{3}$ of the desktop, and becomes active/topmost. * **Activate:** clicking a window, or `F6` (next) / `Shift+F6` (previous), brings it to front and focuses its editor. Only one window is "active" (white frame) at a time. * **Move:** drag the title bar with the mouse, **or** keyboard "move/size mode" via `Ctrl+F5`: then arrows move the window, `Shift+arrows` resize, `Enter` / `Esc` exits the mode. * **Resize:** drag the bottom-right corner with the mouse, or use the size mode above. * **Maximize/restore:** the `[▲]` button, or `Ctrl+F10`. Maximized = fills the whole desktop region. * **Close:** the `[■]` button, or `Ctrl+F4`. If the buffer is modified, raise the "Save changes?" message box (§6.6) first. * **Z-order** is managed by winman; the active window has the highest Z-index.

Framework mapping. `winman.NewWindowManager()`. For each document:

```
ed := NewEditor(buf)           // custom editor primitive (§6.4)
w  := wm.NewWindow().
    Show().
    SetRoot(ed).
    SetDraggable(true).
    SetResizable(true).
    SetTitle(buf.DisplayName())
w.AddButton(&winman.Button{Symbol:'■', OnClick: func(){ closeWindow(w) }})
w.AddButton(&winman.Button{Symbol:'▲', Alignment: winman.ButtonRight, OnClick: func(){ toggleMaximize(w) }})
```

(Exact `winman.Button` field names: confirm against the installed version; adapt symbols/handlers accordingly.)

Acceptance. ≥ 5 windows can be open, overlapping, individually moved/resized/maximized/closed; active (magenta title) vs inactive (grey title) framing is correct; `F6` cycles; closing a dirty buffer prompts to save.

Rendered target — **maximized** window (fills the desktop; title bar shows the restore `[▼]` button; `Ctrl+F10` toggles back):

```
File Edit Search Window Help
[■] [1] HELLO.BAS [▼]

' HELLO.BAS - maximized window (Ctrl+F10 to restore)

DECLARE SUB Greet (name AS STRING)

CLS
PRINT "What is your name?";
INPUT nm$
CALL Greet(nm$)
END

SUB Greet (name AS STRING)
PRINT "Hello, " + name + "!"
END SUB

F1=Help F2=Save F6=Window Ctrl+F10=Restore Alt+X=Exit Ln 8 Col 16
```

The main screen (§5) shows the **floating / overlapping** state: an active window with a magenta title bar in front of an inactive window with a grey title bar.

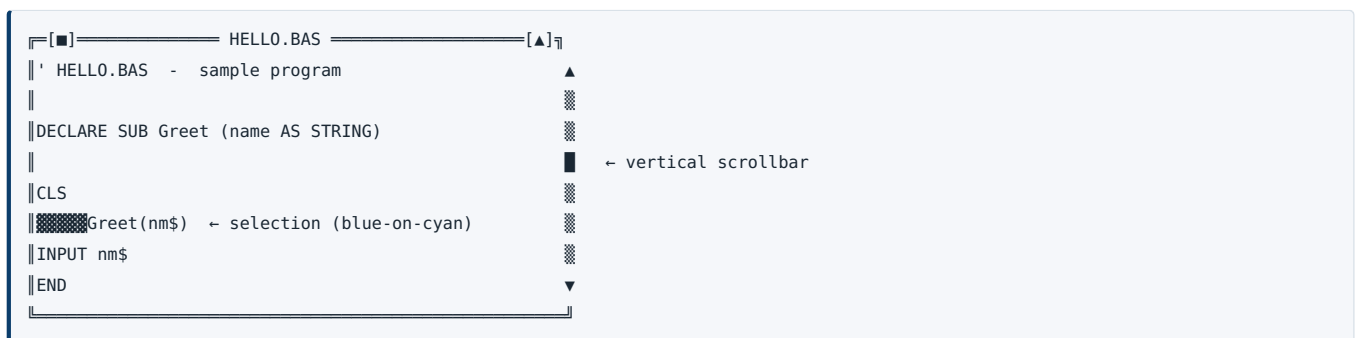
6.4 The editor (core custom primitive)

Purpose. The editable text surface inside each window. This is the heart of the app.

Document/buffer model. See §9. The editor edits one `*Buffer`.

View model. The editor maintains: `topLine`, `leftCol` (scroll offsets), `cursorLine`, `cursorCol`, optional `selAnchor` (line,col) when selecting, and an `overwrite bool` (INS/OVR). It renders only the visible window of the buffer.

Rendering. White text on blue. Tabs expand to the configured tab width (default 4) for display. The cursor cell is drawn Black-on-White. A selection range is drawn Blue-on-Cyan. Long lines clip at the right edge and the horizontal scrollbar appears.



Rendered target (selection + cursor + scrollbar visible): see [images/01_main.png](#).

Editing behaviour. * Insert printable characters at the cursor; in overwrite mode, replace the cell. * `Enter` splits the line (auto-indent to match the previous line's leading whitespace — keep simple, optional but recommended). * `Backspace` / `Delete` with the usual semantics, including joining lines at boundaries. * `Tab` inserts a tab (or N spaces if "use spaces" is on); with a multi-line selection, `Tab` / `Shift+Tab` indent/outdent the block. * Selection via `Shift` + movement keys; replacing typed text deletes the selection first. * Clipboard: maintain an internal clipboard; `Ctrl+C/X/V`. **Stretch:** also sync with the OS clipboard. * Undo/redo: a bounded undo stack of edit operations; `Ctrl+Z` / `Ctrl+Y`.

Movement keys. See §8.

Framework mapping. Implement as a **custom primitive** embedding `tview.Box` (full control of cell drawing and key handling). Do **not** rely on `tview.TextArea` for v1 — you need cell-exact cursor/selection rendering, scrollbars, and the DOS colour treatment, which a custom primitive gives cleanly. (`TextArea` may be evaluated later as an internal engine, but the spec assumes a custom one.)

Acceptance. Type, navigate, select, cut/copy/paste, undo/redo, scroll both axes; INS/OVR toggles; cursor and selection render exactly per the colour table; status bar Ln/Col updates live.

6.5 Status / function-key bar

Purpose. Bottom-row hint bar + live cursor/state readout.

Layout. Light-gray, full width. Left/center: context-sensitive key hints, each as a Red key label + Black `=description`. Right-aligned: `Ln <n> Col <n>` and an `INS / OVR` indicator; a `*` if the active buffer is modified.

Global default set:

```
F1=Help  F2=Save  F3=Open  F6=Window  F10=Menu  Alt+X=Exit      Ln 8  Col 16
```

Menu-navigation set (while a menu is open):

```
F1=Help  ↑=Move  Enter=Select  Esc=Cancel
```

Dialog set (while a dialog is open):

```
Tab=Next  Enter=OK  Esc=Cancel
```

Behaviour. The hint set swaps with context (editing / menu / dialog / window-move mode). Ln/Col/INS update on every cursor change.

Framework mapping. Custom primitive `StatusBar` embedding `tview.Box`; a `SetContext(ctx)` method swaps the hint list; `SetCursor(ln,col,ins)` updates the right side.

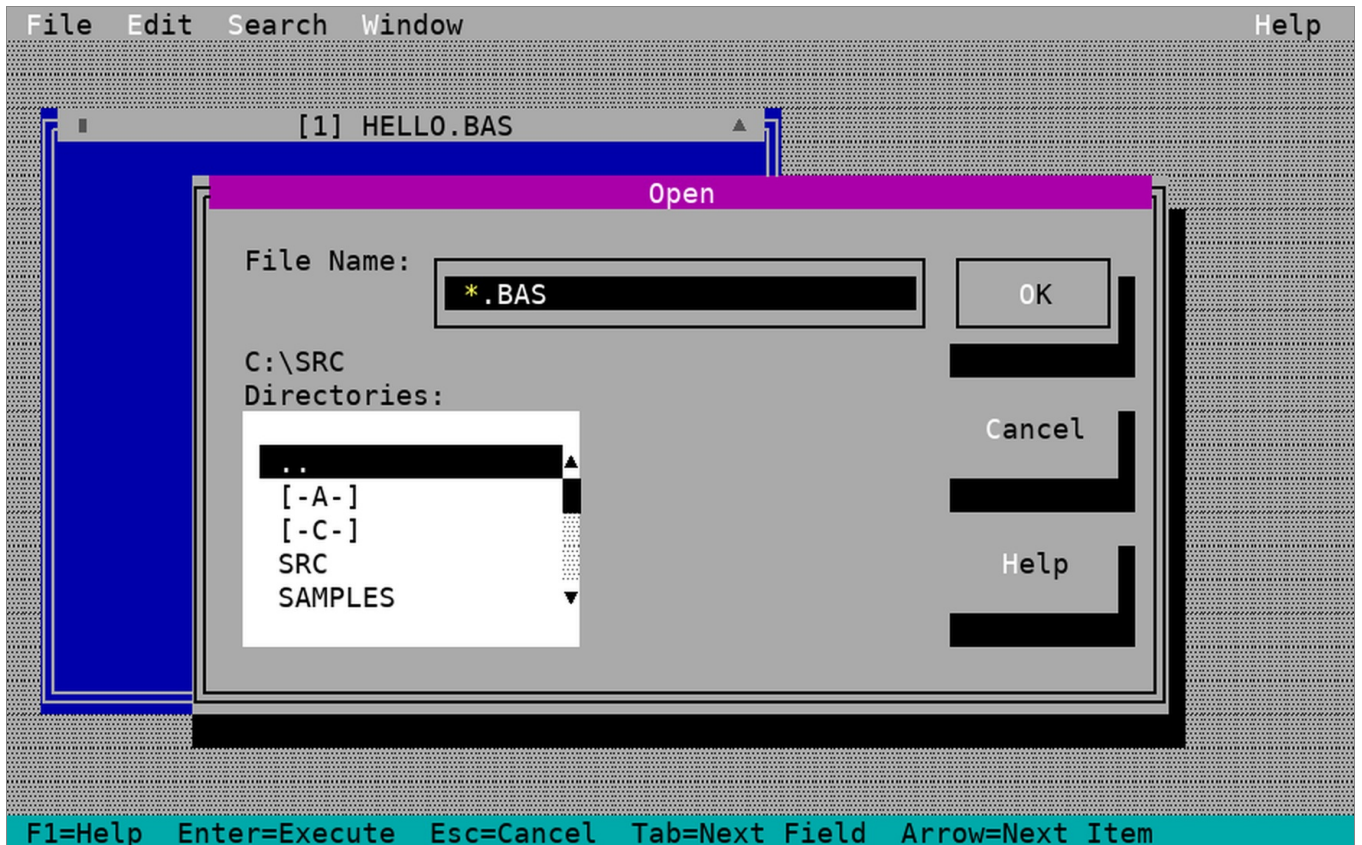
Acceptance. Bar reflects context and live cursor; labels coloured per the table.

6.6 Dialogs (modal)

All dialogs are modal, centered, and double-framed, with a **magenta title bar (white centered text)**, a **light-gray body**, and a **solid-black drop shadow**. The command buttons (`OK` / `Cancel` / `Help`) are **stacked vertically down the right-hand side**, each a grey 3-D button with a black shadow on its right + bottom; the **default button** additionally has a black border drawn around it (focus). Input fields are **white-on-black** with a recessed single-line frame; list boxes are **black-on-white** with a vertical scrollbar. Tab/Shift+Tab cycle focus; the default/focused button fires on `Enter`; `Esc` cancels. (This matches the VBDOS "New Form" dialog used as the template for all of these.)

Dialogs required for v1:

1. **Open** — `File Name:` input (with `*.BAS` default filter), a current-path line (`C:\SRC`), a `Files` list and a `Directories` list (white, scrollable), and `OK` / `Cancel` / `Help` buttons down the right. Navigating into a dir refreshes the lists. Selecting a file + OK opens it in a new window.



2. **Save As** — directory navigation + file-name input + `OK/Cancel`. Confirms overwrite if the file exists.
3. **Message box** — used for "Save changes to ? [Yes] [No] [Cancel]", error messages, and confirmations. Variable button set.
4. **Find** — "Find What" input, `[ ] Match case`, `[ ] Whole word`, `OK/Cancel`.
5. **Replace** — "Find What" + "Replace With" inputs, the same checkboxes, `Find Next` / `Replace` / `Replace All` / `Cancel`.
6. **About** — static product box with `OK`.

Framework mapping. `tview.Form` inside a `winman` modal window (or `tview.Modal` for the simple message box). Buttons use the default-button colour treatment. Lists use `tview.List`. Center the modal in the desktop region and draw the shadow behind it.

Acceptance. Each dialog opens modally (input to windows blocked), keyboard-navigates, returns the right result, and disposes cleanly restoring focus.

7. Menu tree (full command set)

Mnemonics underlined here by **bold**; accelerators in the right column. Items beyond the editor scope are intentionally absent (no Run/Debug menus).

File

Item	Accel	Action
N ew		Create empty buffer in a new window
O pen...	F3	Open dialog → new window
S ave	F2	Save active buffer (prompts Save As if untitled)
Save A s...		Save As dialog
Save A ll		Save every modified buffer
—		
C lose	Ctrl+F4	Close active window (prompt if dirty)
P rint...		(Stretch) write buffer to a file/printer; may be disabled/greyed in v1
—		
E xit	Alt+X	Quit (prompt for each dirty buffer)

Edit

Item	Accel	Action
Undo	Ctrl+Z	
Redo	Ctrl+Y	
—		
Cut	Ctrl+X / Shift+Del	
Copy	Ctrl+C / Ctrl+Ins	
Paste	Ctrl+V / Shift+Ins	
Delete	Del	Delete selection
—		
Select All	Ctrl+A	

Search

Item	Accel	Action
Find...	Ctrl+F	Find dialog
Find Next	F3* (or Ctrl+L)	Repeat last find (note: if F3=Open globally, use Ctrl+L for Find Next)
Replace...	Ctrl+H	Replace dialog
Go to Line...	Ctrl+G	Prompt for line number

Resolve the F3 conflict in code: keep **F3 = Open** (classic QB) and bind **Find Next = Ctrl+L**. State the final choice in the status bar/help.

Window

Item	Accel	Action
Next	F6	Activate next window
Previous	Shift+F6	Activate previous window
—		
Cascade		Offset-stack all windows
Tile	F5	Tile all windows to fill desktop
Maximize/Restore	Ctrl+F10	Toggle active window maximize
Move/Size	Ctrl+F5	Enter keyboard move/size mode
—		
(window list)	1..9	Numbered list of open windows; selecting activates

Help

Item	Accel	Action
Keys		Show the keyboard map (a read-only window or dialog)
About...		About dialog

7.4 Optional stretch: BASIC syntax highlighting

Off by default. If enabled (Options or a `--highlight` flag): colour BASIC keywords (`SUB PRINT DECLARE IF THEN...`) Yellow, strings Light-Cyan, comments (`' / REM`) Light-Gray, line numbers Brown — over the blue background. Must be a pure display layer that never alters the buffer. Keep this strictly optional so it doesn't compromise the "editor only" scope.

8. Keyboard map

Primary scheme is **CUA** (the VBDOS/Windows style). Classic WordStar-style chords (`Ctrl+K...`) are an optional alternate set; CUA is canonical.

Global

Key	Action
F1	Help / Keys
F2	Save
F3	Open
F5	Tile windows
F6 / Shift+F6	Next / previous window
F10 / Alt	Activate menu bar
Alt+F / Alt+E / Alt+S / Alt+W / Alt+H	Open File/Edit/Search/Window/Help menu
Ctrl+F4	Close active window
Ctrl+F5	Window move/size mode
Ctrl+F10	Maximize/restore active window
Alt+1 ... Alt+9	Activate window N
Alt+X	Exit

Editor — cursor movement

Key	Action
← → ↑ ↓	Char/line movement
Ctrl+← / Ctrl+→	Previous / next word
Home / End	Start / end of line
Ctrl+Home / Ctrl+End	Start / end of document
PgUp / PgDn	Page up / down
Ctrl+PgUp / Ctrl+PgDn	Scroll horizontally / to far columns

Editor — selection & editing

Key	Action
Shift + any movement key	Extend selection
Ctrl+A	Select all
Ctrl+C / Ctrl+Ins	Copy
Ctrl+X / Shift+Del	Cut
Ctrl+V / Shift+Ins	Paste
Del	Delete char / selection
Backspace	Delete left / selection
Tab / Shift+Tab	Insert tab / (with selection) indent-outdent block
Ins	Toggle insert/overwrite
Ctrl+Z / Ctrl+Y	Undo / redo
Ctrl+F / Ctrl+L / Ctrl+H	Find / Find next / Replace
Ctrl+G	Go to line

Menu / dialog navigation

Key	Action
↑ ↓ ← →	Move within menus / between fields
Enter	Select / default button
Esc	Close dropdown → close menu → cancel dialog
Tab / Shift+Tab	Next / previous control in a dialog
mnemonic letter	Jump to menu item / dialog control

9. Document & file model

```

type Buffer struct {
    Lines    []string // one entry per line, no trailing newline stored
    Path      string  // "" if untitled
    Title     string  // "Untitled1" if Path == ""
    Modified  bool
    EOL       string  // "\n" (LF) or "\r\n" (CRLF), preserved from load; new = LF
    TabWidth  int     // display only; default 4
    UseSpaces bool    // if true, Tab inserts spaces
}

```

- **Encoding:** UTF-8 on disk and in memory. (Optional stretch: detect/convert CP437 for authentic `.bas` files.)
- **Untitled buffers** get `Untitled1`, `Untitled2`, Saving prompts for a path.
- **Modified flag** drives the `*` prefix in titles, the status-bar `*`, and save prompts on close/exit.
- **Line endings:** detect on load, preserve on save; new files use LF.
- **Large files:** acceptable to load fully into memory for v1; lines as a slice is fine.

10. Project structure

```

dosedit/
├─ go.mod
├─ main.go           # wire app: screen, palette, menubar, statusbar, winman, first window
├─ internal/
│  ├─ theme/
│  │  └─ palette.go    # the 16 tcell colors + named style helpers (§4.2–4.3)
│  │  └─ ui/
│  │     ├─ menubar.go # custom MenuBar primitive + dropdown overlay
│  │     ├─ statusbar.go # custom StatusBar primitive (context-aware)
│  │     ├─ editor.go   # custom Editor primitive (Draw + InputHandler)
│  │     ├─ window.go   # winman window creation + title-bar buttons + active framing
│  │     └─ dialogs.go  # Open / SaveAs / MessageBox / Find / Replace / About
│  └─ buffer/
│     ├─ buffer.go     # Buffer model + load/save + line ops
│     └─ undo.go       # undo/redo stack
└─ app/
   ├─ app.go          # App struct: holds wm, windows[], clipboard, key router
   ├─ commands.go     # newWindow/open/save/close/find/etc. (menu + key actions)
   └─ keys.go         # global key routing & accelerator table
└─ testdata/
   └─ HELLO.BAS

```

Keep `theme` and `buffer` free of UI imports so they're unit-testable.

11. Milestones (build in this order)

Each milestone must compile and run.

1. **M1 — Shell.** App boots; palette defined; textured **grey** desktop; static menu bar (no dropdowns) and static **cyan** status bar; Alt+X quits restoring the terminal.
2. **M2 — One window + editor typing.** winman integrated; one editor window centered; type/insert/backspace/enter; cursor + scroll; Ln/Col in status bar; INS/OVR toggle.
3. **M3 — Editor completeness.** Selection (Shift+move), clipboard cut/copy/paste, undo/redo, Home/End/word/doc movement, Tab/indent, horizontal scroll + scrollbars.
4. **M4 — Menus.** Custom menu bar with dropdowns, mnemonics, accelerators, Esc semantics, drop shadows; wire File/Edit/Search/Window/Help actions to existing commands (stub dialogs allowed).
5. **M5 — MDI.** Multiple windows: New, cascade offset, F6 cycling, active/inactive framing, move/size (mouse + Ctrl+F5), maximize (Ctrl+F10), close (Ctrl+F4) with dirty prompt, Tile (F5), window list / Alt+1..9.
6. **M6 — Dialogs & files.** Open / Save As (with dir navigation), Save / Save All, Message box (save-changes), Find / Replace / Go to Line, About. Real file I/O end-to-end.

7. **M7 — Polish.** Resize reflow & window clamping; mouse on menus (stretch); cross-compile binaries; help/keys screen; optional syntax highlight flag.
-

12. Build & run

```
# scaffold
go mod init dosedit
go get github.com/gdamore/tcell/v2
go get github.com/rivo/tview
go get github.com/epiclabs-io/winman      # MDI windows (or r3ap3r2004/winman fork)

# run
go run .

# build current OS
go build -o dosedit .

# cross-compile
GOOS=windows GOARCH=amd64 go build -o dosedit.exe .
GOOS=darwin  GOARCH=arm64 go build -o dosedit-macos .
GOOS=linux   GOARCH=amd64 go build -o dosedit-linux .
```

Terminal notes for authentic colours. The app sets RGB explicitly, so it needs a truecolor-capable terminal. On Linux/macOS that's the default modern terminal. On Windows use **Windows Terminal** (not legacy conhost) for correct RGB and box-drawing glyphs. Document this in the README.

13. Definition of done (acceptance checklist)

- [] Boots to the master layout; one empty `Untitled1` window focused.
 - [] Colours match §4.2–4.3 exactly (verify against `images/*.png`).
 - [] Menu bar: all menus open via mouse, F10/Alt, and Alt+letter; mnemonics + accelerators work; Esc semantics correct; dropdowns cast shadows.
 - [] ≥ 5 overlapping windows; each moves, resizes, maximizes, closes; active framing is white/double, inactive is gray; F6/Shift+F6 cycle; Tile & Cascade work.
 - [] Editor: full movement set, selection, cut/copy/paste, undo/redo, Tab indent, INS/OVR, both scrollbars; cursor & selection rendered per the colour table; Ln/Col live.
 - [] Open/Save/Save As/Save All/Close round-trip files; dirty buffers prompt before close/exit; titles show `*` when modified.
 - [] Find / Replace / Replace All / Go to Line work.
 - [] Resizing the terminal reflows bars and clamps windows without corruption.
 - [] Single static binary builds for Linux, macOS, Windows; runs in Windows Terminal.
-

Appendix A — main.go wiring skeleton (illustrative)

```
package main

import (
    "github.com/gdamore/tcell/v2"
    "github.com/rivo/tview"
    "github.com/epiclabs-io/winman"
    "dosedit/internal/theme"
    "dosedit/internal/ui"
    "dosedit/internal/app"
)

func main() {
    theme.Init() // define the 16 RGB colors as tcell.Color vars

    tapp := tview.NewApplication().EnableMouse(true)
```

```

wm := winman.NewWindowManager()

desktop := ui.NewDesktop(wm) // paints texture, embeds wm
menubar := ui.NewMenuBar()
status := ui.NewStatusBar()

root := tview.NewFlex().SetDirection(tview.FlexRow).
    AddItem(menubar, 1, 0, false).
    AddItem(desktop, 0, 1, true).
    AddItem(status, 1, 0, false)

a := app.New(tapp, wm, menubar, status) // holds windows[], clipboard, commands
a.WireMenus() // populate menu tree -> command callbacks
a.NewUntitledWindow() // first window

tapp.SetInputCapture(a.RouteGlobalKeys) // $8 accelerators/menu activation

if err := tapp.SetRoot(root, true).SetFocus(desktop).Run(); err != nil {
    panic(err)
}
_ = tcell.KeyEsc
}

```

Appendix B — custom primitive shape (MenuBar / StatusBar / Editor)

```

type Editor struct {
    *tview.Box
    buf      *buffer.Buffer
    topLine  int
    leftCol  int
    curLine  int
    curCol   int
    selAnchor *Pos // nil when no selection
    overwrite bool
}

func NewEditor(b *buffer.Buffer) *Editor {
    e := &Editor{Box: tview.NewBox(), buf: b}
    return e
}

// Draw paints visible lines, selection (blue-on-cyan), cursor (black-on-white),
// and scrollbars within e's inner rect. Use screen.SetContent(x,y,r,nil,style).
func (e *Editor) Draw(screen tcell.Screen) {
    e.Box.DrawForSubclass(screen, e)
    x, y, w, h := e.GetInnerRect()
    _ = x; _ = y; _ = w; _ = h
    // ... render buffer window, selection, cursor, scrollbars ...
}

// InputHandler handles editor keys ($8 editor section); unhandled keys bubble up.
func (e *Editor) InputHandler() func(ev *tcell.EventKey, setFocus func(p tview.Primitive)) {
    return e.WrapInputHandler(func(ev *tcell.EventKey, setFocus func(p tview.Primitive)) {
        switch ev.Key() {
            case tcell.KeyLeft: e.moveLeft(ev.Modifiers())
            case tcell.KeyRight: e.moveRight(ev.Modifiers())
            case tcell.KeyRune: e.insertRune(ev.Rune())
            // ... etc ...
        }
    })
}

```

`MenuBar` and `StatusBar` follow the same shape: embed `tview.Box`, override `Draw` to paint their single row(s) with the §4.3 styles, and (for `MenuBar`) override `InputHandler` for navigation; the open dropdown is a second primitive layered above the desktop via `tview.Pages`.

Appendix C — palette helper sketch

```
package theme

import "github.com/gdamore/tcell/v2"

var (
    Black    = tcell.NewRGBColor(0,0,0)
    Blue     = tcell.NewRGBColor(0,0,170)
    Green    = tcell.NewRGBColor(0,170,0)
    Cyan     = tcell.NewRGBColor(0,170,170)
    Red      = tcell.NewRGBColor(170,0,0)
    LGray    = tcell.NewRGBColor(170,170,170)
    DGray    = tcell.NewRGBColor(85,85,85)
    LBlue    = tcell.NewRGBColor(85,85,255)
    Magenta  = tcell.NewRGBColor(170,0,170)
    Yellow   = tcell.NewRGBColor(255,255,85)
    White    = tcell.NewRGBColor(255,255,255)
    ShadowBg = Black // dialog/menu/button shadow is solid black
)

// Style helpers
func Desktop() tcell.Style { return tcell.StyleDefault.Foreground(Black).Background(LGray) } // texture '⌘'
func MenuNormal() tcell.Style { return tcell.StyleDefault.Foreground(Black).Background(LGray) }
func MenuSelect() tcell.Style { return tcell.StyleDefault.Foreground(White).Background(Black) } // reverse video
func EditorText() tcell.Style { return tcell.StyleDefault.Foreground(White).Background(Blue) }
func Selection() tcell.Style { return tcell.StyleDefault.Foreground(Blue).Background(Cyan) }
func ActiveFrame() tcell.Style { return tcell.StyleDefault.Foreground(White).Background(Blue) }
func ActiveTitle() tcell.Style { return tcell.StyleDefault.Foreground(White).Background(Magenta) }
func InactiveTitle() tcell.Style { return tcell.StyleDefault.Foreground(Black).Background(LGray) }
func StatusBar() tcell.Style { return tcell.StyleDefault.Foreground(Black).Background(Cyan) }
func DialogTitle() tcell.Style { return tcell.StyleDefault.Foreground(White).Background(Magenta) }
func InputField() tcell.Style { return tcell.StyleDefault.Foreground(White).Background(Black) }
```

End of specification. The ASCII mockups in §5–§6 are the authoritative layout reference; the PNGs in [images/](#) show the target colours. Implement in the milestone order of §11.